EX27 · CLASS_17_VECTORS · 02 / PYTHON REFERENCE GUIDE

NumPy cross products

Functions, inputs, outputs and short examples; no full exercise solution.

Learning sequence
01 Math theory → 02 Python reference → 03 Exercise → 04 Pseudocode → Programming.

Prerequisites: EX25 and EX26, NumPy arrays, Euclidean norm and dot product.

1. Before starting

Use a single Python file for EX27. Import NumPy as in EX25 and EX26:

```
import numpy as np
```

Create a 3D vector from a list. Components must be ordered x, y, z.

```
v = np.array([1.0, 2.0, 3.0])
print(v.shape)       # (3,)
print(v[0])          # 1.0
```

2. np.cross(a, b)

Element	Description
Input a	First NumPy 3D vector.
Input b	Second NumPy 3D vector.
Returns	Three-component array: vector a × b.
Order	a × b differs from b × a.

```
a = np.array([1.0, 0.0, 0.0])
b = np.array([0.0, 1.0, 0.0])
c = np.cross(a, b)
print(c)             # [0. 0. 1.]
```

`np.cross` returns a vector, while `np.dot` returns a scalar for two 1D vectors.

2D warning
For planar problems, explicitly set z = 0. This ensures three-component arrays and avoids version-dependent behaviour of 2D cross products.

3. np.dot(a, b)

Takes compatible arrays and returns the scalar dot product when both are 1D vectors. We can use it to check the result is perpendicular.

```
u = np.array([1.0, 0.0, 0.0])
v = np.array([0.0, 0.0, 1.0])
print(np.dot(u, v))  # 0.0
```

4. np.linalg.norm(v)

Returns the Euclidean norm of a 1D vector (a non-negative scalar). For a cross product it gives the magnitude.

```
v = np.array([0.0, 3.0, 4.0])
magnitude = np.linalg.norm(v)
print(magnitude)     # 5.0
```

5. Components: shape and indexing

The `.shape` attribute tells you the array dimensions. Use `v[0]`, `v[1]`, `v[2]` for x, y, z.

```
v = np.array([4.0, -2.0, 6.0])
print(v.shape)       # (3,)
print(v[2])          # 6.0
```

6. How to validate a result

First compute a cross product manually. Then check perpendicularity with `np.dot` and compare magnitudes against manual arithmetic. Floating-point results can be close to, rather than exactly, zero.

```
x = np.array([1.0, 0.0, 0.0])
y = np.array([0.0, 1.0, 0.0])
z = np.cross(x, y)
print(np.dot(z, x))  # 0.0
print(np.dot(z, y))  # 0.0
```

7. Common errors

Situation	How to avoid it
Swapped arguments	Use `np.cross(r, F)` for moments.
Using * on arrays	* is elementwise multiplication, not a × b.
Missing z component	Write [x,y,0] for planar problems.
Missing units	Moment in N·m; not N or J.

8. Mini cheat sheet

```
np.array([x, y, z])   # vector 3D
np.cross(a, b)        # vector perpendicular
np.dot(a, b)          # producto escalar / dot product
np.linalg.norm(a)     # magnitud / magnitude
a.shape               # forma / shape
```